

スタックとキュー

1. はじめに：なぜ「データの整理術」を学ぶのか

プログラミングの学習を始めると、必ず「アルゴリズム」と「データ構造」という言葉に出会います。文系出身の方の中には「なんだか数学的で難しそうだ」と感じる方もいるかもしれませんが、安心してください。これらは、実務における「効率性」を支える極めてシンプルな知恵なのです。

これを料理に例えてみましょう。「アルゴリズム」はおいしい料理を作るための手順、つまり「レシピ」です。対して「データ構造」は、食材を入れておくための「ボウル」や「棚」に相当します。

いくら優れたレシピがあっても、食材がバラバラに置かれていたり、取り出しにくい容器に入っていたりすれば、調理（プログラムの実行）に時間がかかり、効率が落ちてしまいます。データの「持ち方（形）」を工夫することで、実行速度を上げ、メモリの消費を抑えることができるのです。適切なデータ構造の選択は、「スマートなプログラミング」への第一歩となります。

今回は、データの扱い方の基本ルールとして、私たちの日常生活にも深く根ざしている「スタック」と「キュー」を解き明かしていきます。

2. スタック（Stack）：後入れ先出し（LIFO）の魔法

まず紹介するのは「スタック」です。この構造の核心は、**LIFO（Last-In, First-Out：後入れ先出し）** というルールにあります。

基本概念と比喻

スタックは「トレイの山」や「積み上げられた本」をイメージしてください。私たちはトレイを積むとき、常に一番上に置きますね。そして、使うときも一番上にある（＝最後に置いた）ものから手に取ります。

主要な操作と可視化

スタックの操作には専用の呼び名があります。

1. Push（プッシュ）：データをスタックの一番上に「入れる」操作。
2. Pop（ポップ）：一番上にあるデータを「出す」操作。

【スタックのイメージ】

```
[ 3 ] ← TOP（ここからPop / ここにPush）  
[ 2 ]  
[ 1 ]
```

★アドバイス：ここが「つまずき」ポイント！

初学者がやりがちなミスが、空っぽのスタックからデータを出そうとすることです。これを「スタック・アンダーフロー」と呼び、プログラムが強制終了する原因になります。出す前には必ず「中身があるか？」を確認するのがプロの作法ですよ。

実務での応用：再帰と「戻る」機能

スタックはコンピュータの心臓部で大活躍しています。

- 関数の呼び出し（再帰）：プログラムがある処理の中で別の処理（関数）を呼び出す際、コンピュータは「今の場所」をスタックに保存します。これは読書中の「しおり」のようなものです。処理が終わるとスタックから場所を取り出し（Pop）、元の位置に正確に戻ることができます。
- Undo（元に戻す）：テキストエディタの「元に戻す」やブラウザの「戻る」ボタンは、直前の動作をスタックに保存しているため、常に「最新の履歴」から順に遡ることができるのです。

3. キュー（Queue）：先入れ先出し（FIFO）のルール

スタックが「後に入れたものを先に」扱うのに対し、より公平な順番待ちを実現するのが「キュー」です。

基本概念と比喻

キューの核心は、**FIFO（First-In, First-Out：先入れ先出し）**です。これは「レジの行列」や「チケット購入の列」そのものです。

主要な操作と可視化

キューでは、データの入り口（末尾）と出口（先頭）が分かれています。

- Enqueue（エンキュー）：列の最後尾に「並ぶ（データを追加する）」操作。
- Dequeue（デキュー）：列の先頭から「抜ける（データを取り出す）」操作。

【キューのイメージ】

```
出口(Exit) ← [ 1 ][ 2 ][ 3 ] ← 入口(Entrance)
                (Dequeue)      (Enqueue)
```

実務での応用：順番待ちの管理

キューは、システムの安定性と「公平性」を守るために不可欠です。

- プリントジョブ：オフィスの共有プリンタを想像してください。もしプリンタがスタック構造だったら、最後に印刷ボタンを押した人の書類が一番先に印刷されてしまいます。これでは不公平ですね。だからこそ、先に頼んだ順に処理するキューが必要なのです。
- OSのタスク管理：コンピュータが複数の処理を抱えた際、受け付けた順に実行するためにキューが使われます。

4. スタック vs キュー：決定的な違いと使い分け

これら2つの構造に優劣はありません。「用途に応じた最適解」を選ぶのがエンジニアの仕事です。

比較まとめ表

項目	スタック (Stack)	キュー (Queue)
概念	LIFO（後入れ先出し）	FIFO（先入れ先出し）
操作名	Push / Pop	Enqueue / Dequeue
データの出入り	同じ端（TOP）からのみ	片方の端から入り、反対から出る
代表的な比喻	トレイの山、積み上げた本	レジの行列、チケットの列
主な用途	Undo機能、再帰、ブラウザの「戻る」	プリント待ち、OSの実行待ち行列

「So What?（だから何?）」レイヤー：選択を誤ると... 例えば、最新のメールから順に読みたいシステムで「キュー」を選んでしまったらどうなるでしょう。ユーザーは最新の連絡を読むために、過去の全メールをスクロールして一番下まで行かなければなりません。目的が「最新優先」ならスタック、「順番遵守」ならキュー。この判断が使い勝手を左右します。

5. プログラミングにおける実装のヒント：構造体の活用

さて、概念を理解したところで、これらをどうコードに落とし込むかを考えましょう。

構造体（Struct）でデータをセットにする

CS50の講義でも紹介されるように、データは「値」だけでなく「関連情報」とセットで扱うと便利です。例えば、電話帳を作るとき、名前（string）と電話番号（string）をバラバラの配列で管理すると、並び順がズレた瞬間に電話番号が他人のものになってしまうリスクがあります。

そこで、C言語の typedef struct を使って「人（person）」という独自の型を作ります。

```
typedef struct {
    string name;    // 名前
    string number;  // 電話番号
} person;
```

このように構造体を使えば、スタックやキューの中に「名前と番号がセットになったデータ」を安全に蓄積できるようになり、プログラムの読みやすさ（可読性）と直しやすさ（保守性）が格段に向上します。

配列で実装する際の「インデックス」の罠

スタックやキューを「配列」で自作する際、文系学生が最も間違いやすいのが「何番目から数えるか」です。

6. まとめ：データ構造を味方につける

今回学んだ内容は、単なる暗記項目ではなく、エンジニアとしての「思考の道具」です。

- スタック：最新を優先するLIFO。コンピュータの記憶（再帰）や履歴管理の要。
- キュー：順番を重んじるFIFO。公平な処理待ちを実現する守護者。
- データ構造の選択：アルゴリズム（レシピ）を活かすためのボウル選びが、効率を決定する。

今回学んだ基礎は、将来的に「木構造 (Tree)」や「グラフ (Graph)」といった複雑な構造、さらにはNIST (米国国立標準技術研究所) が定めるような次世代の暗号技術 (ポスト量子暗号など) を理解する上での強固な土台となります。まずは簡単なコードを書いて、データがPushされたりEnqueueされたりする様子を体感してみてください。それが、計算効率を意識できるプロフェッショナルへの第一歩です。